



Tema 9. Segmentacion - vbnmn

Estructura de Computadores (Universidad Rey Juan Carlos)

ESTRUCTURA DE COMPUTADORES

Camino de datos y control: segmentación

Luis Rincón / José Luis Bosque / Óscar David Robles / José San Martín



Programa

- 1. Introducción.**
- 2. Concepto de segmentación.**
- 3. Ejecución de instrucciones con segmentación.**
- 4. Diseño de repertorios de instrucciones para segmentación.**
- 5. Riesgos en la segmentación.**
- 6. Segmentación del camino de datos.**
- 7. Técnicas avanzadas de segmentación.**
- 8. Conclusiones e implicaciones del diseño segmentado.**



1. Introducción

Hasta ahora se han analizado diversas maneras de implementar el camino de datos y el control:

- Implementación unicycle.
- Implementación multiciclo con control cableado.

Implementación unicycle:

- Las instrucciones se estructuran en fases.
- Todas las instrucciones tardan un único ciclo de reloj.
- El periodo de reloj es largo: se ajusta a la instrucción más lenta $\Rightarrow$ ineficiencia.

Implementaciones multiciclo:

- Las instrucciones se dividen en fases.
- Cada fase se ejecuta en un único ciclo de reloj.
- El periodo de reloj se ajusta a la fase más lenta.
- Algunas instrucciones son más rápidas que en la implementación unicycle, y otras son más lentas.
- Pega: en cada ciclo de reloj hay muchos elementos inactivos en el camino de datos.



Introducción

La segmentación trata de hacer posible las ventajas de ambos enfoques:

- Cada instrucción tarda varios ciclos en completarse.
- Hay una instrucción completada por cada ciclo de reloj.
- El ciclo de reloj es corto.
- Todos los elementos del camino de datos trabajan en casi todos los ciclos.

Para ello:

- Las instrucciones se dividen en fases.
- Las instrucciones se ejecutan en forma solapada.

Objetivos básicos del diseñador para obtener las máximas prestaciones:

- Hacer trabajar todos los elementos del camino de datos lo más posible.
- Hacer el ciclo de reloj corto: la duración de todas las fases deberá ser similar.
- Conseguir que la circuitería necesaria para el control tenga un impacto poco significativo en la duración del ciclo de reloj.

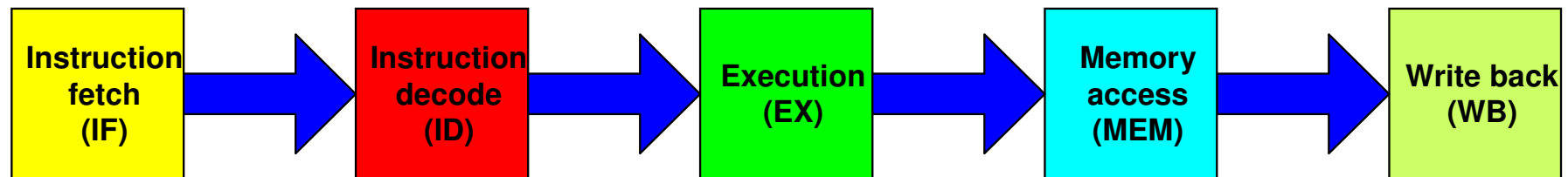
Dificultades:

- A veces varias instrucciones compiten por el mismo recurso simultáneamente.
- Pueden surgir problemas de sincronización al acceder a los datos.
- Es complicado implementar las instrucciones de salto y bifurcación.



2. Concepto de segmentación

Fases de ejecución de las instrucciones



Instruction Fetch (IF): lectura de la instrucción en memoria.

Instruction Decode (ID): decodificación de la instrucción.

Execution (EX): ejecución de la operación o cálculo de la dirección de un operando residente en memoria.

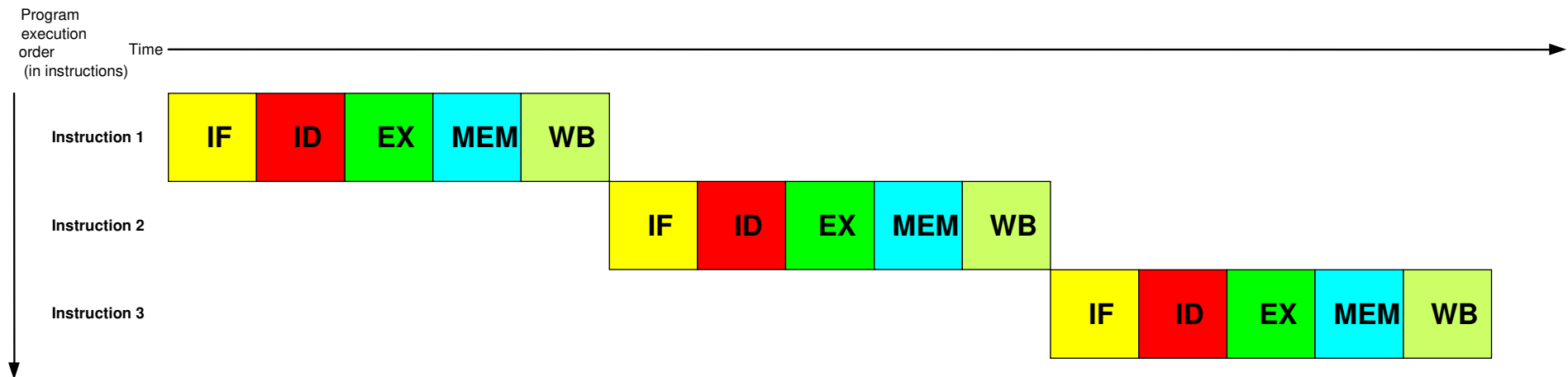
Memory Access (MEM): acceso a un operando residente en memoria.

Write Back (WB): escritura de resultados.

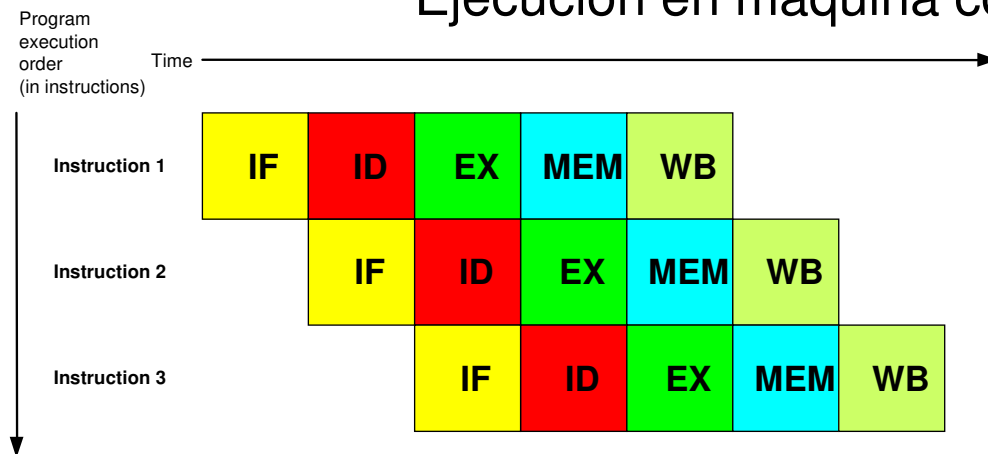


Concepto de segmentación

Ejecución en máquina sin segmentación



Ejecución en máquina con segmentación





Concepto de segmentación

La segmentación (***pipelining***) es una técnica de implementación de caminos de datos que permite solapar la ejecución de operaciones (instrucciones).

Para ello:

- Las instrucciones se dividen en fases.
- El camino de datos se estructura en etapas como si fuera una cadena de montaje (***pipeline***: cañería, cauce).
- En cada ciclo de reloj una instrucción ocupa una única fase (como en la implementación multiciclo).
- Cuando termina un ciclo de reloj, cada instrucción pasa a la siguiente fase.
- En cada ciclo de reloj, se inicia la lectura (*fetch*) de una instrucción.
- En cada ciclo de reloj, una instrucción sale de la cadena (termina su ejecución).

Por tanto, en un instante determinado T:

- En cada etapa del camino de datos hay una instrucción en proceso.
- Cada etapa procesa una instrucción distinta.
- Todas las etapas se encuentran ocupadas.



Concepto de segmentación

⇒ La latencia de todas las instrucciones es idéntica.
(Latencia: tiempo de ejecución).

⇒ Ninguna instrucción se ejecuta más rápido que con las implementaciones uniciclo o multiciclo.

Sin embargo:

¡Los programas se ejecutan mucho más rápido que en las implementaciones uniciclo y multiciclo!

¿Por qué? ⇒ PORQUE LA PRODUCTIVIDAD ES MUCHO MAYOR

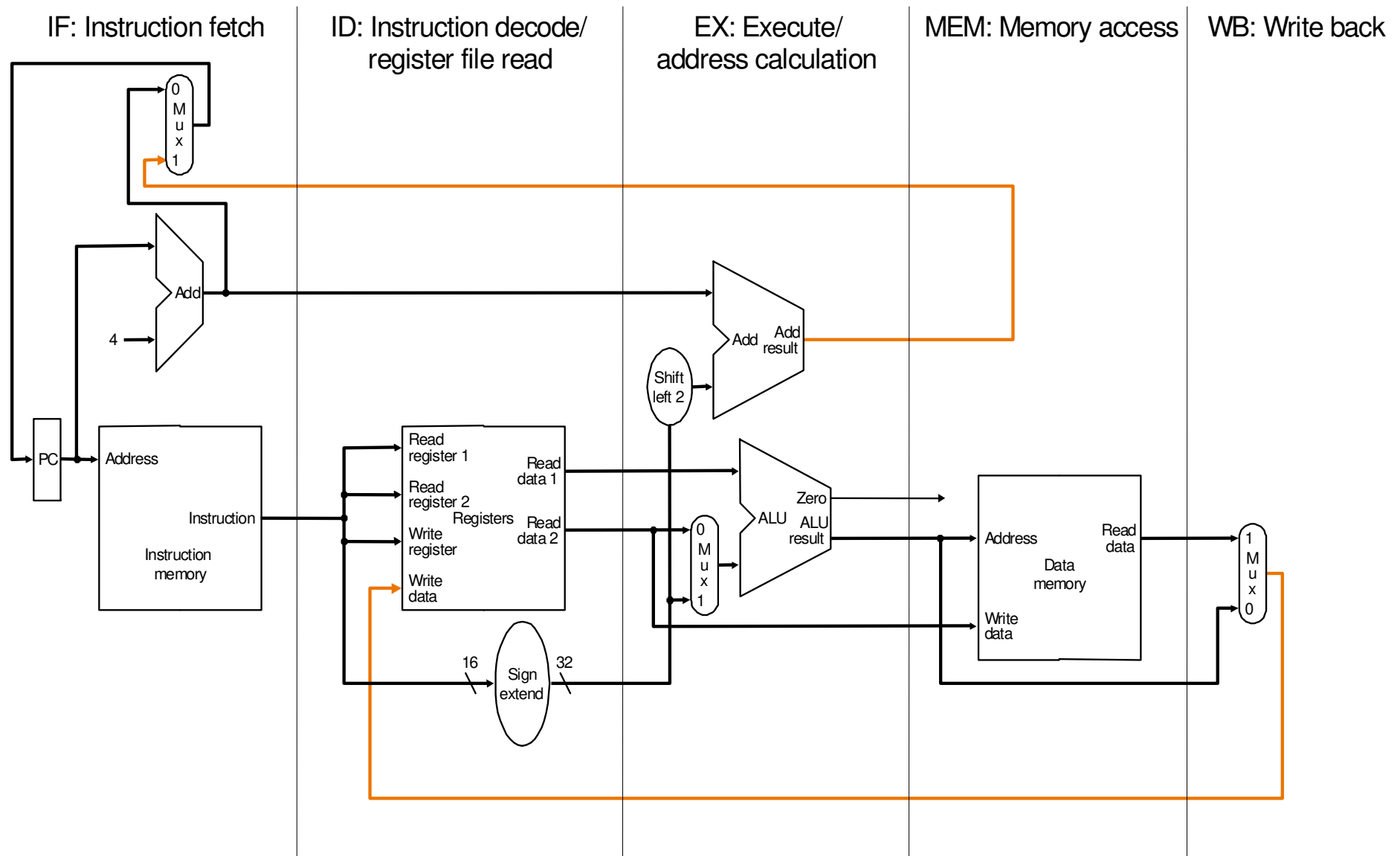
(Productividad = número de tareas realizadas / tiempo consumido en realizarlas).

⇒ Con la segmentación:

- Se completa una instrucción por ciclo (como en la implementación uniciclo).
- El ciclo de reloj es corto (como el de la implementación multiciclo).



3. Ejecución de instrucciones y segmentación





Fases de ejecución de las instrucciones

Instrucción	Búsqueda de instrucción en memoria IF	Lectura registros ID	Operación UAL / cálculo de dirección EX	Acceso a dato en memoria MEM	Escritura registros WB	Total
LW	2 ns	1 ns	2 ns	2 ns	1 ns	8 ns
SW	2 ns	1 ns	2 ns	2 ns		7 ns
Tipo R: ADD, SUB, AND, OR, SLT	2 ns	1 ns	2 ns		1 ns	6 ns
BEQ	2 ns	1 ns	2 ns			5 ns

En la máquina unicycle, el ciclo de reloj es de 8 ns.

En la máquina multiciclo, el ciclo de reloj es de 2 ns (duración de la etapa más lenta).

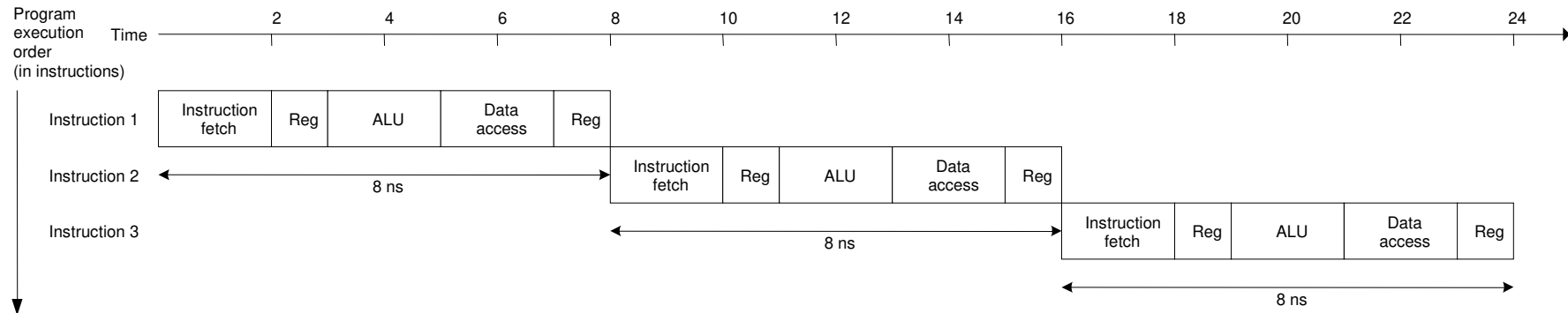
En la máquina segmentada, el ciclo de reloj será de 2 ns (duración de la etapa más lenta).

- Las etapas de la segmentación no están perfectamente equilibradas.

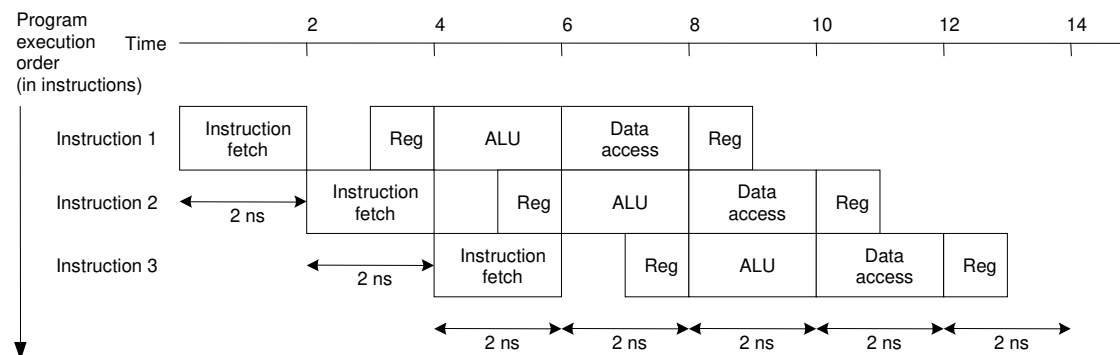


Ejecución de instrucciones y segmentación

Ejecución en máquina unicycle



Ejecución con segmentación



Accesos al banco de registros

Lecturas: 2ª mitad del ciclo.

Escrituras: 1ª mitad del ciclo.



Ejecución de instrucciones y segmentación

La aceleración alcanzada será:

$$A_{n_i} = \frac{\text{Tiempo sin segmentación}}{\text{Tiempo con segmentación}} = \frac{t_{\text{noseg}} * n_i}{t_{\text{seg}} * (n_e - 1 + n_i + \text{burbujas en la ejecución del programa})}$$

donde

- t_{noseg} : tiempo medio de ejecución por instrucción en una máquina no segmentada.
- t_{seg} : duración del ciclo de reloj en la máquina segmentada.
- n_i : número de instrucciones ejecutadas
- n_e : número de etapas de la segmentación

En nuestro caso, la secuencia de las tres instrucciones tarda lo siguiente:

- Máquina unicycle: 24 ns.
- Máquina segmentada: 14 ns.

Aceleración: **A** = 1,71

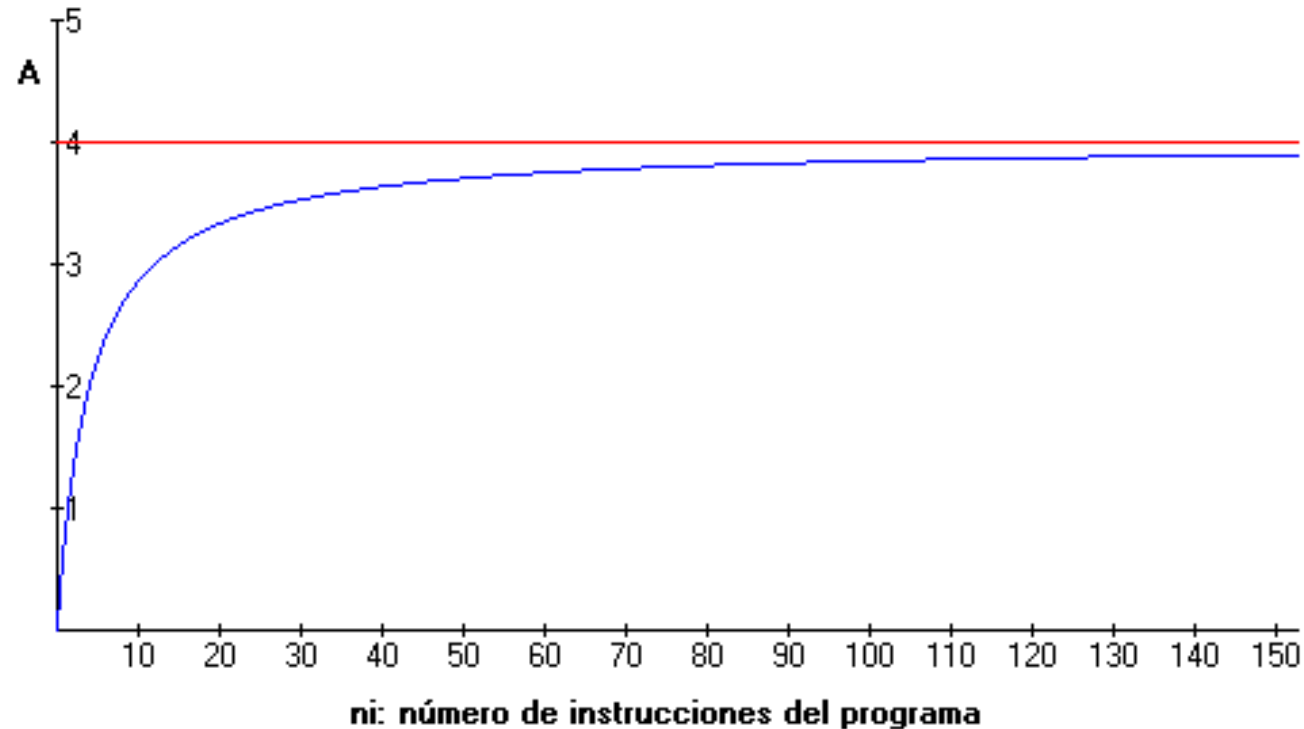


Ejecución de instrucciones y segmentación

Al aumentar el número de instrucciones ($n_i \rightarrow \infty$)

$$A_{\infty} = \lim_{n \rightarrow \infty} \frac{t_{\text{noseg}} * n_i}{t_{\text{seg}} * (n_e - 1 + n_i)} = \frac{t_{\text{noseg}}}{t_{\text{seg}}}$$

En nuestro caso, como máximo tendremos que $A_{\infty} \rightarrow 4$.



Si las etapas de la segmentación estuviesen equilibradas:

$$t_{\text{seg}} = \frac{t_{\text{noseg}}}{n_e}$$

En tal caso, si $n_i \rightarrow \infty$, entonces $A \rightarrow n_e$.



4. Repertorio de instrucciones y segmentación

Para que haya una buena segmentación es recomendable que:

1. La longitud de todas las instrucciones sea la misma.
2. El repertorio de instrucciones cuente con pocos formatos distintos.
3. Todo campo presente en varios formatos distintos ocupe la misma posición en todos ellos si es posible.
4. Las instrucciones se puedan dividir en un número de fases homogéneo.
5. Todas las fases duren aproximadamente lo mismo.
6. Las instrucciones tengan direccionamientos fijos o que siempre tarden lo mismo.
7. Los datos residentes en memoria estén alineados a palabra.
8. Los datos en memoria sólo participen en cargas o almacenamientos.

El repertorio de MIPS está especialmente diseñado para segmentación.



5. Riesgos en la segmentación

Son situaciones que producen que la siguiente instrucción no se pueda ejecutar en el siguiente ciclo.

Tipos de riesgos (*hazards*) en la segmentación

Riesgos estructurales: la circuitería no puede soportar la combinación de instrucciones que se pretenden ejecutar en el mismo ciclo, puesto que varias instrucciones compiten simultáneamente por el mismo recurso *hardware*.

Riesgos de datos: surgen cuando una instrucción usa como operando el resultado de otra instrucción previa que no ha sido aún escrito por ésta en destino.

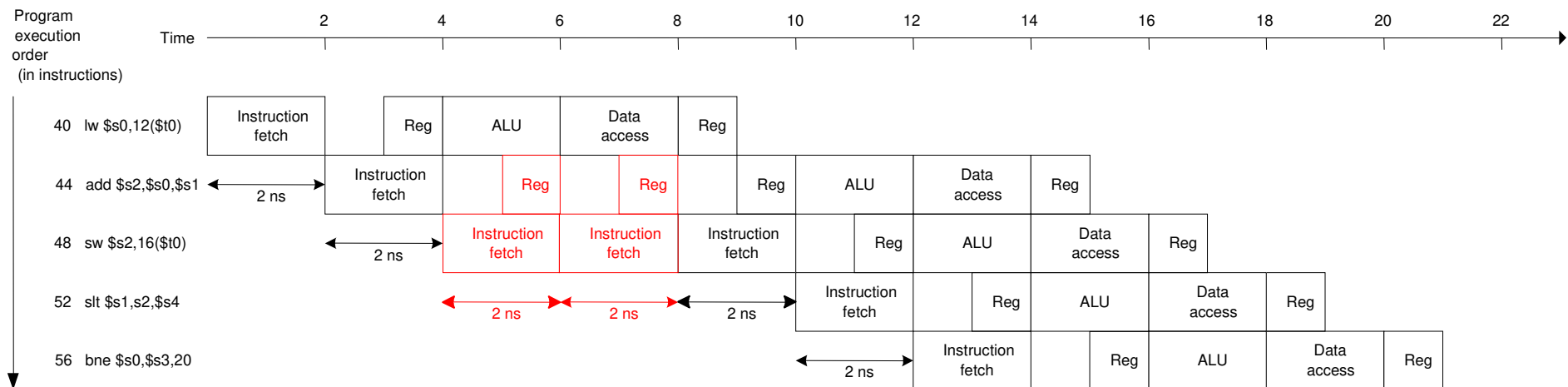
Riesgos de control: surgen cuando hay instrucciones que rompen la secuencia de ejecución del programa (saltos o bifurcaciones).



Una situación de riesgo en la segmentación provoca que las instrucciones no puedan ejecutarse normalmente.

Un riesgo puede provocar un bloqueo de la segmentación (*pipeline stall*).

- Las instrucciones no pueden avanzar por el cauce.
- Se produce una burbuja (*bubble*) en la ejecución de instrucciones.



Dependiendo del tipo de riesgo, pueden aplicarse diversas técnicas para reducir el tamaño de la burbuja o incluso eliminarla.



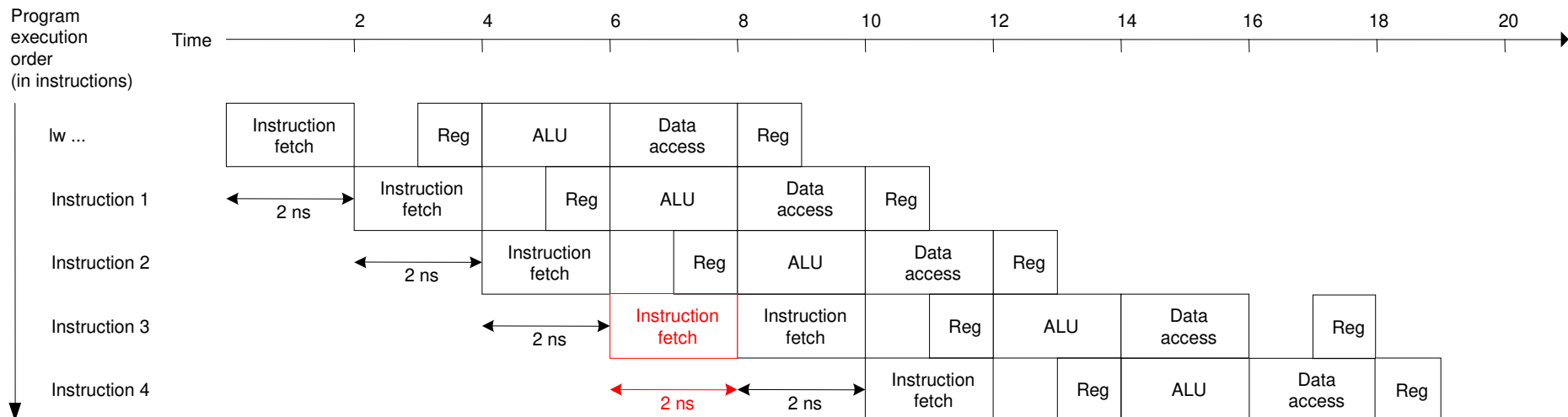
Riesgos estructurales en la segmentación

Surgen cuando varias instrucciones presentes en el cauce intentan usar el mismo recurso simultáneamente.

Dependen de:

- El diseño del camino de datos del procesador.
- El repertorio de instrucciones.

Ejemplo: acceso simultáneo a memoria en dos etapas distintas de la instrucción.



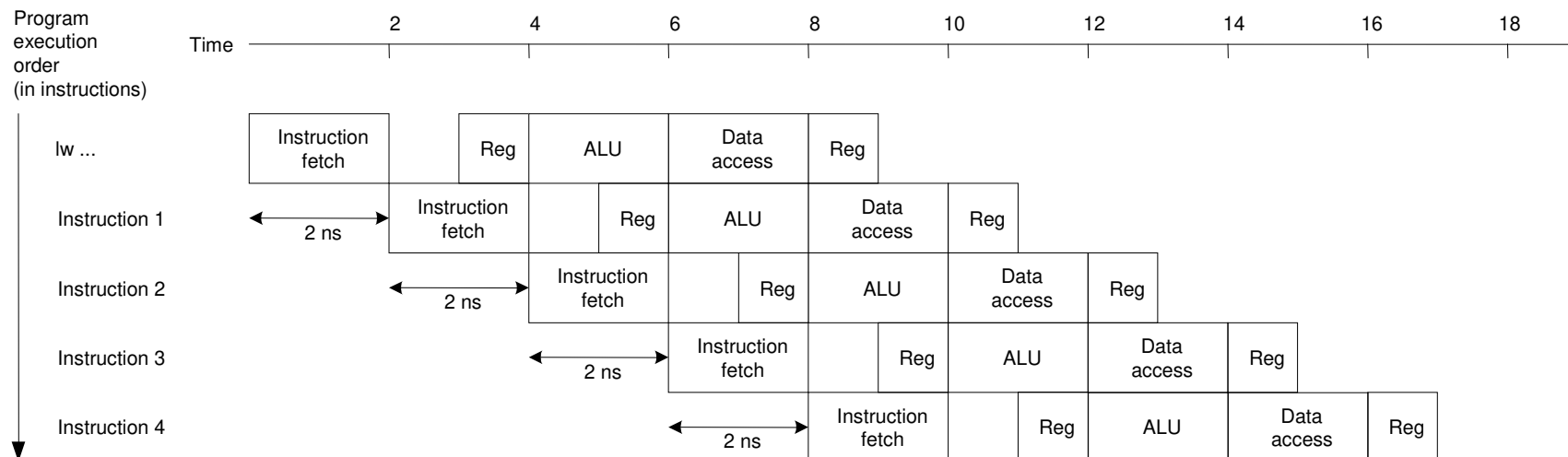


Soluciones a los riesgos estructurales

Para resolver un riesgo estructural hay varias posibilidades:

- Introducir una burbuja en el cauce.
- Fraccionar el ciclo de reloj para permitir varios usos del elemento en un único ciclo (ejemplo: banco de registros).
- Replicar el elemento que produce el riesgo para evitar la burbuja (ejemplo: memorias separadas para instrucciones y datos).

En nuestro camino de datos tenemos memoria de instrucciones y de datos separadas, y no habría conflictos.





Dependencias de datos

Si dos o más
instrucciones
comparten un dato

```
add $1, $2, $3  
sub $3, $4, $5  
...
```

Dependencia
de datos

- RAW
- WAR
- WAW
- RAR



Dependencias de datos

Dependencia RAW (*read after write*): una instrucción tiene que leer un dato escrito por otra instrucción previa.

```
add    $s0,$s1,$s2
sub    $s5,$s0,$s3
```

Dependencia WAR (*write after read*): una instrucción lee un valor escrito por otra instrucción posterior.

```
sub    $s5,$s0,$s3
add    $s0,$s1,$s2
```

Dependencia WAW (*write after write*): dos instrucciones escriben sobre un mismo dato.

```
add    $s0,$s1,$s2
sub    $s0,$s5,$s3
```

Dependencia RAR (*read after read*): dos instrucciones leen un mismo dato fuente.

```
add    $s1,$s0,$s2
sub    $s5,$s0,$s3
```



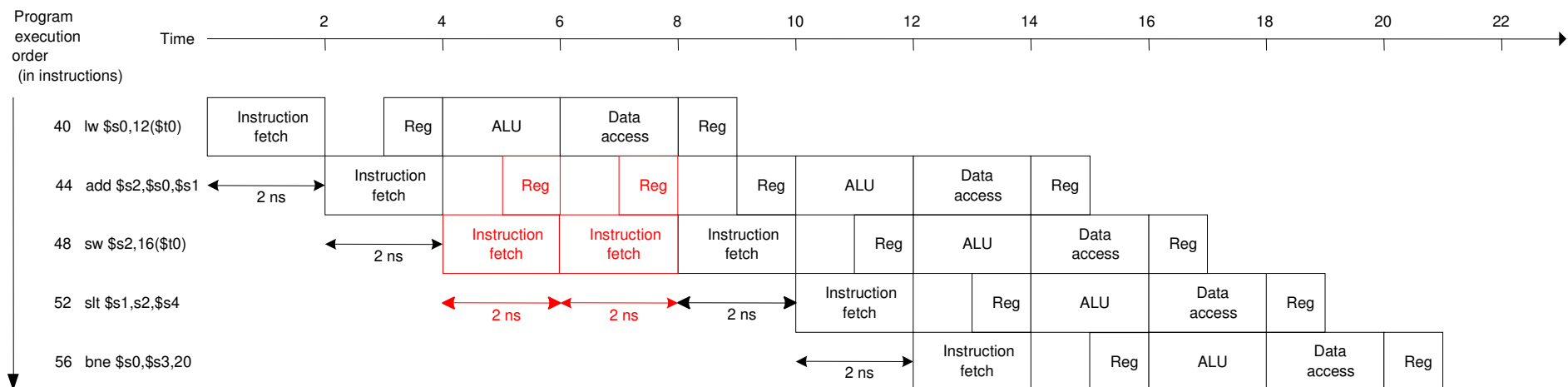
Riesgos de datos en la segmentación

Los riesgos de datos suceden cuando una instrucción tiene como operando fuente el resultado de otra instrucción que aún está en el cauce y no ha completado su ejecución.

Nuestro problema está en las dependencias RAW (leer después de escribir).

- Los registros se escriben en la etapa WB.
- Los datos en memoria en instrucciones de almacenamiento se escriben en la etapa MEM.

Posible solución: bloquear el cauce.



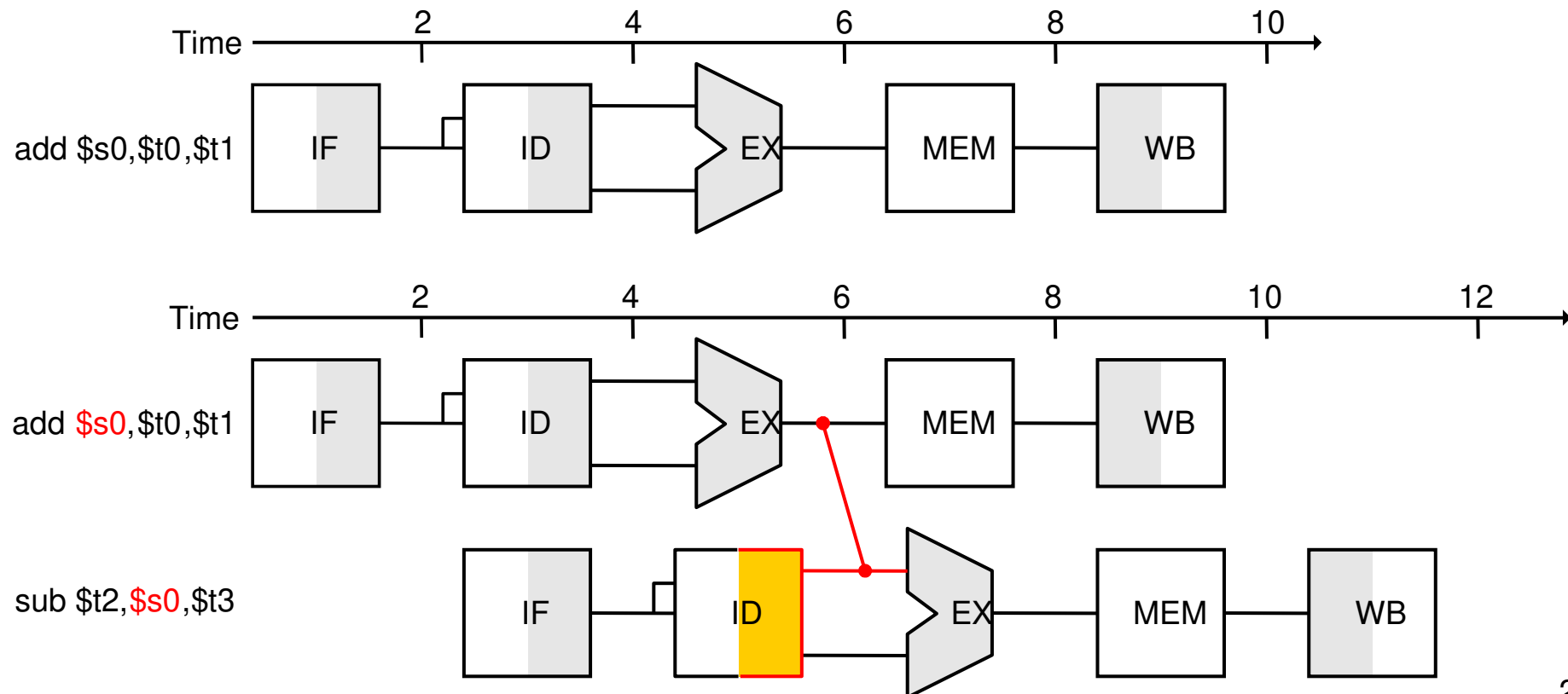


Riesgos de datos: anticipación

La técnica de anticipación, adelantamiento o cortocircuito (*forwarding*) permite poner a disposición de etapas previas de la segmentación resultados obtenidos por etapas posteriores.

- Para ello se establecen cortocircuitos que interconectan las unidades del camino de datos como sea necesario.

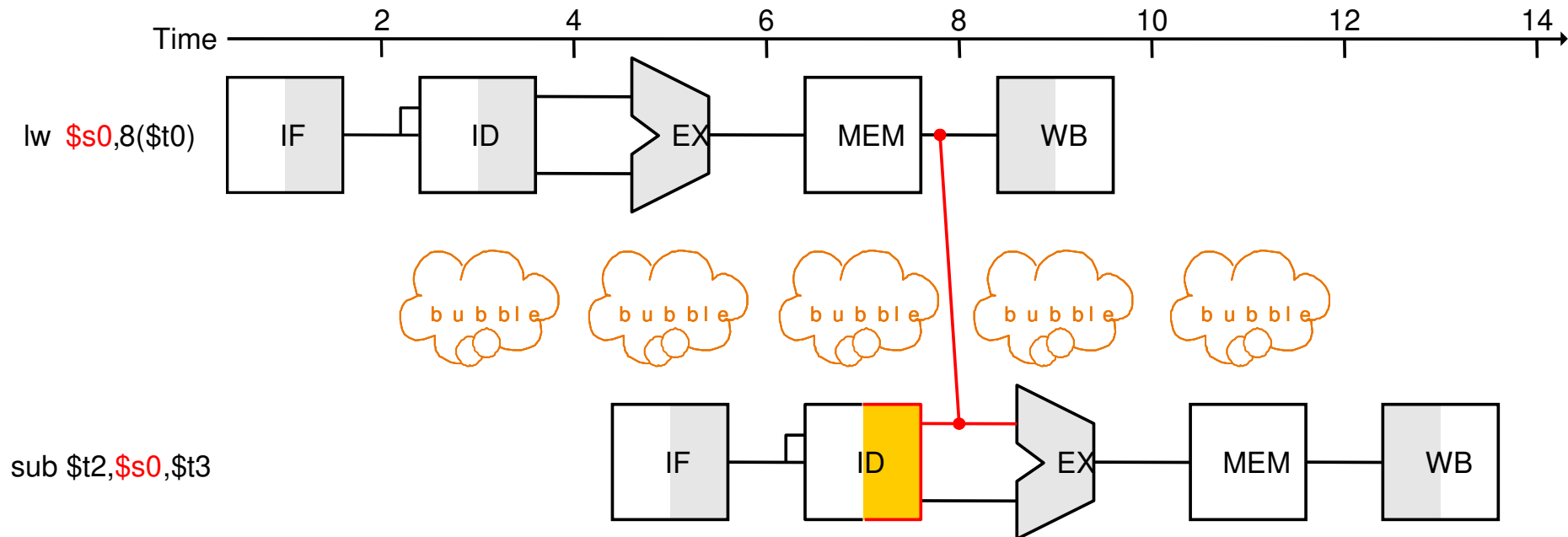
Ejemplo





Riesgos de datos en la segmentación

Aun utilizando adelantamiento, en ocasiones no es posible eliminar totalmente la burbuja.

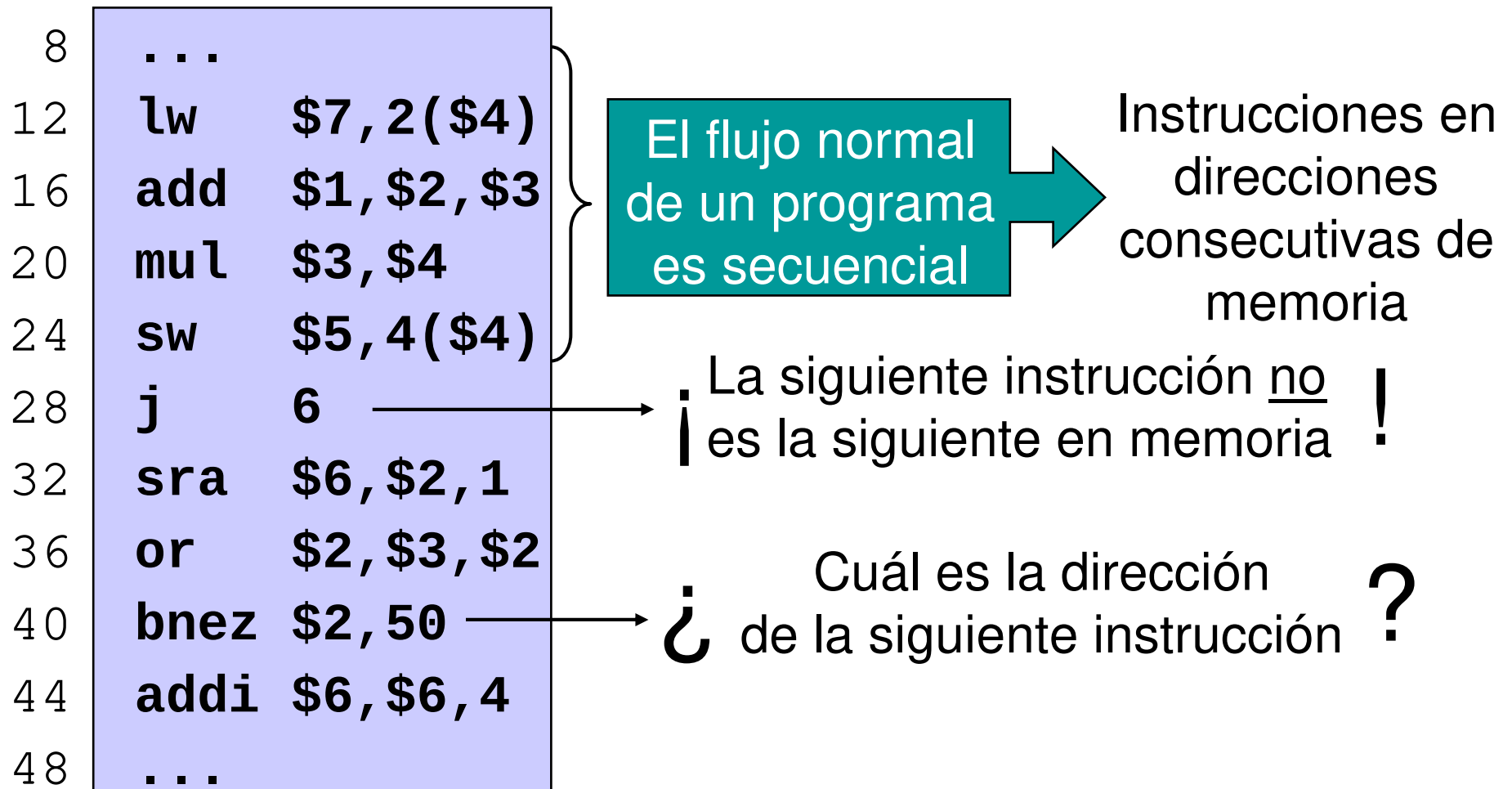


Es necesario que el hardware sea capaz de detectar los riesgos de datos y bloquear el cauce si es preciso.

El hueco de la burbuja puede ser rellenado con una instrucción independiente (**carga retardada**).



Riesgos de control en la segmentación





Riesgos de control en la segmentación

Surgen cuando hay instrucciones que rompen la secuencia de ejecución del programa (saltos o bifurcaciones).

Ante un salto o bifurcación puede haber dos caminos posibles:

- La siguiente instrucción es el destino del salto (**salto efectivo**).
- La siguiente instrucción va detrás en la secuencia (**salto no efectivo**).

El objetivo es determinar cuál es la próxima instrucción lo antes posible.

- Al mismo tiempo que la instrucción de salto entra en el segundo ciclo, una nueva instrucción entra en el cauce, y ¡en ese instante la instrucción de salto aún no ha sido decodificada! $\Rightarrow$ ES IMPOSIBLE SABER A TIEMPO CUÁL ES LA SIGUIENTE INSTRUCCIÓN, SEA EL SALTO O RAMIFICACIÓN EFECTIVO O NO.

Información de salto

- **Valor de la condición de salto:** indica si el salto es efectivo o no.
- **Dirección de salto:** indica cuál es la siguiente instrucción si el salto es efectivo.



Salto incondicional en MIPS

Instrucciones:

- **j** objetivo
- **jal** objetivo
- **jalr** rs, rd
- **jr** rs

Valor de la condición de salto:

- Los saltos siempre son efectivos.

Dirección de salto:

- En **j** y **jal**: $PC = PC_{31..28} \parallel (\text{objetivo} \ll 2)$
- En **jr** y **jalr**: $PC = rs$



Ramificación condicional en MIPS

Instrucciones de ramificación condicional típicas:

- **bgtz rs,etiqueta**: ramifica si $rs > 0$
- **bgez rs,etiqueta** y **bgezal rs,etiqueta**: ramifican si $rs \geq 0$
- **blez rs,etiqueta**: ramifica si $rs \leq 0$
- **bltz rs,etiqueta** y **bltzal rs,etiqueta**: ramifican si $rs < 0$
- **beq rs,rt,etiqueta**: ramifica si $rs=rt$
- **bne rs,rt,etiqueta**: ramifica si $rs \neq rt$

Las instrucciones **bcZf** y **bcZt** ramifican en función del valor de un bit determinado del coprocesador **Z** ($Z=0,1,2$ ó 3).

Dirección de salto:

- El traductor convierte la etiqueta en un desplazamiento relativo al PC.
- La dirección de salto se calcula sumando el desplazamiento al PC previamente incrementado.



Información de salto

Valor de la condición de salto:

- Los saltos incondicionales siempre son efectivos: se sabe en ID.
- En el esquema inicial el valor de la condición de salto en ramificaciones condicionales se conoce en EX.
- Añadiendo circuitería, el valor de la condición de salto en ramificaciones condicionales puede conocerse en ID.

Dirección de salto y decisión sobre si se toma el salto o no:

- Salto incondicional: en ID.
- Ramificación condicional: en el esquema inicial de la segmentación se conoce en EX, y la decisión sobre el salto se toma en la etapa MEM.
- El sumador que calcula la dirección de ramificación se puede mover a la etapa ID.



Solución a los riesgos de control: predicción

Consiste en tomar una decisión anticipada sobre si el salto es efectivo o no, y después corroborarla cuando la información sobre el salto esté disponible.

Si la predicción es errónea, hay que suspender la ejecución de las instrucciones que han entrado en el cauce indebidamente (se crea burbuja).

Posibilidades en la predicción:

Predecir como no efectivo: suponemos que el salto no se produce.

- Si acertamos en la predicción, no hay burbuja.

Predecir como efectivo: suponemos que el salto se produce.

- Aunque acertemos en la predicción se produce burbuja, ya que hay que calcular la dirección de salto y cargarla en el PC.

Predicción dinámica: consiste en predecir en función de la historia pasada del salto.

- Exige mantener tablas *hardware* para poder realizar la predicción (*buffer* de predicción de saltos).

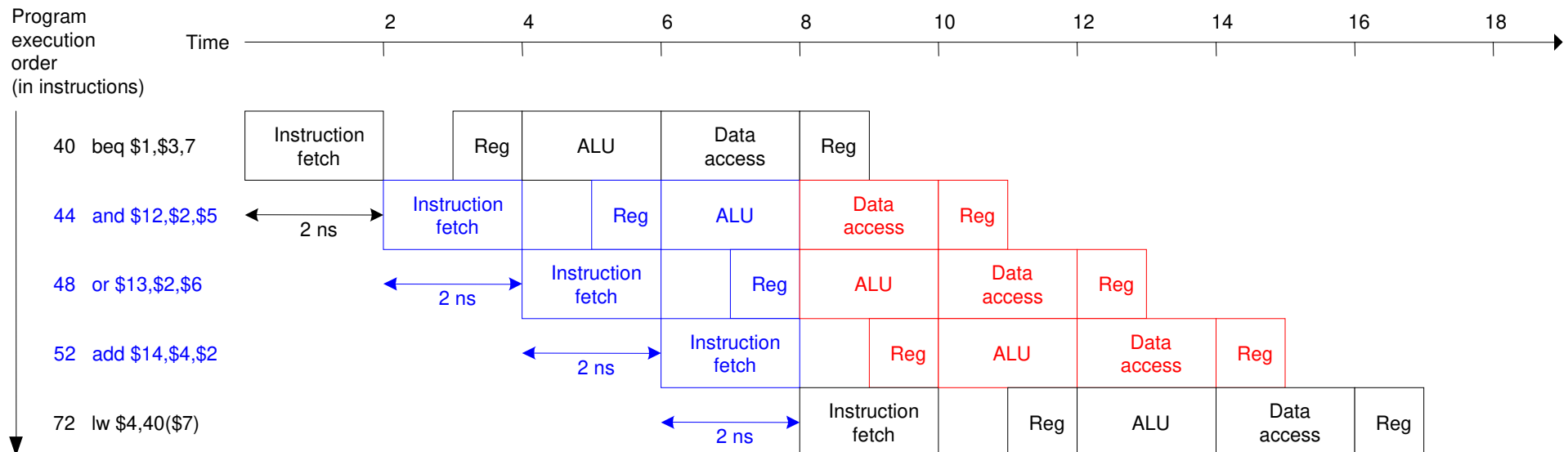


Solución a los riesgos de control: predicción

Ejemplo:

- Predecir no efectivo
- Salto tomado en MEM
- **Predicción errónea**

```
36    sub $10, $4, $8
40    beq $1, $3, 7
44    and $12, $2, $5
48    or  $13, $2, $6
52    add $14, $4, $2
56    slt $15, $6, $7
...
72    lw  $4, 40($7)
```



Si la predicción es correcta, las instrucciones continúan por el cauce normalmente.

Si la predicción es errónea

- Hay que anular las instrucciones que han entrado indebidamente en el cauce.
- Se produce una burbuja de 3 ciclos

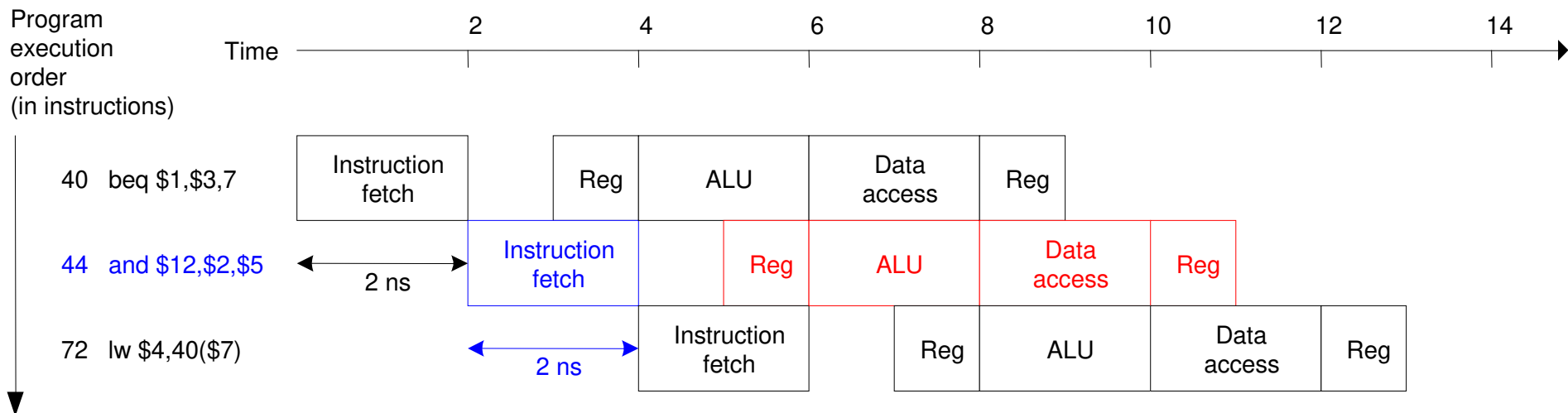


Solución a los riesgos de control: predicción

Ejemplo:

- Predecir no efectivo
- Salto tomado en ID
- **Predicción errónea**

```
36      sub $10, $4, $8
40      beq $1, $3, 7
44      and $12, $2, $5
48      or  $13, $2, $6
52      add $14, $4, $2
56      slt $15, $6, $7
...
72      lw  $4, 40($7)
```



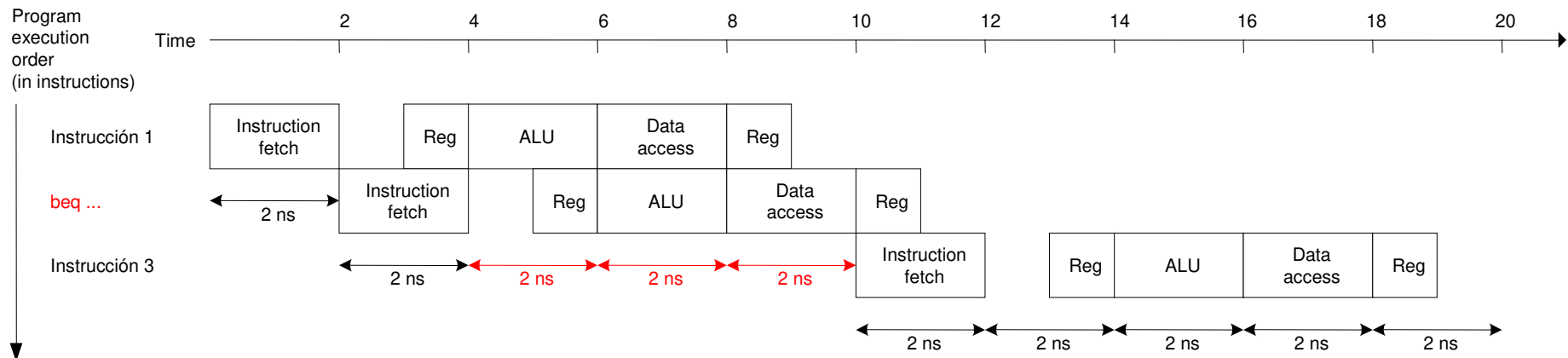
Si la predicción es correcta, las instrucciones continúan normalmente por el cauce.

Si la predicción es errónea

- Hay que anular el efecto de una única instrucción
- Se produce una burbuja de 1 ciclo



Ejemplo:







Solución a los riesgos de control: salto retardado

Estaría bien aprovechar
los huecos de retardo de
los saltos con
instrucciones útiles

Deben ser instrucciones
que se ejecuten SIEMPRE

Salto Retardado

Si un salto
produce h huecos
de retardo →

- El compilador debe insertar h instrucciones detrás del salto.
- El efecto del salto se debe retardar h ciclos.



Solución a los riesgos de control: salto retardado

Consiste en eliminar la burbuja forzando a que las instrucciones que van detrás del salto (o ramificación) se ejecuten completamente.

El efecto del salto se retarda tantos ciclos como sea necesario.

- El número de ciclos de retardo de un salto depende de la fase de la instrucción en la que se conozca la información de salto.

Si la información de salto se conoce en MEM:

- El salto se retarda 3 ciclos.
- Queda un hueco de retardo para 3 instrucciones.

Si la información de salto se conoce en ID:

- El salto se retarda 1 ciclo.
- Queda un hueco de retardo para 1 instrucción.



Solución a los riesgos de control: salto retardado

El salto retardado queda oculto a los ojos del programador.

- Es el traductor de ensamblador quien se encarga de reordenar las instrucciones del programa a fin de rellenar el hueco de retardo.
- Las instrucciones empleadas para rellenar el hueco no deben depender del salto ni influir en él.
- Si el ensamblador no encuentra instrucciones útiles para rellenar el hueco, introduce **nop**.

De esta forma:

- Los huecos que deja la burbuja se rellenan con instrucciones útiles cuya ejecución no dependa del resultado del salto.
- El *hardware* asume que el traductor de ensamblador ha rellenado el hueco de retardo con instrucciones útiles.

A veces es difícil rellenar el hueco con instrucciones útiles, sobre todo si el hueco es grande.

- Es importante reducir el hueco al mínimo número de ciclos posibles



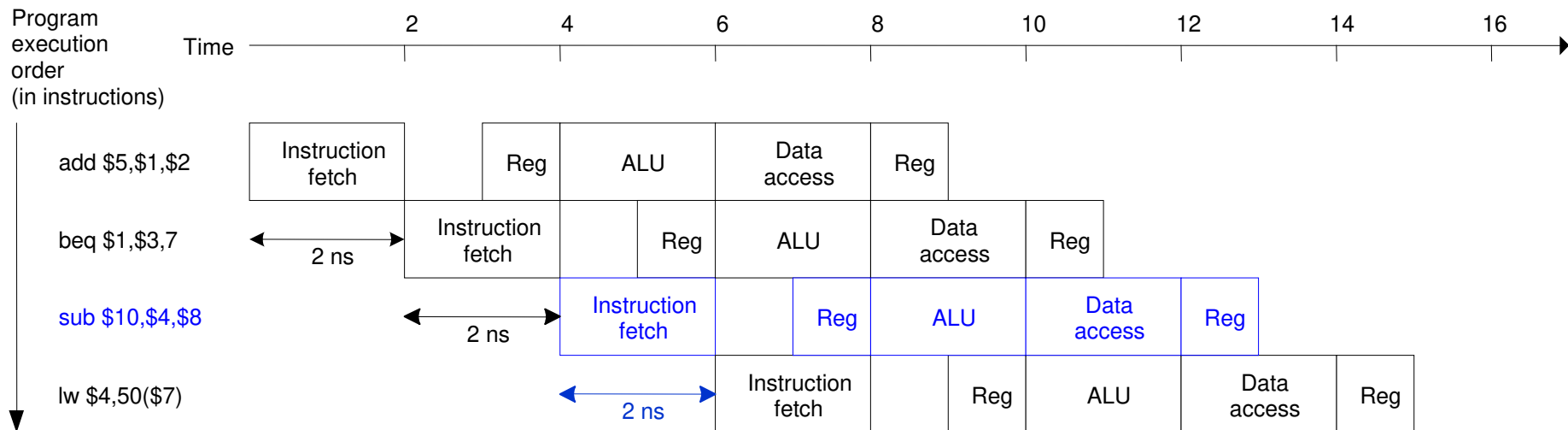
Salto retardado: ejemplo

Rellenamos el hueco con la instrucción anterior.

El traductor intercambia las instrucciones **sub** y **beq** (y modifica el desplazamiento en esta última)

32	add \$5,\$1,\$2
36	sub \$10,\$4,\$8
40	beq \$1,\$3,7
44	and \$12,\$2,\$5
48	or \$13,\$2,\$6
52	add \$14,\$4,\$2
56	slt \$15,\$6,\$7
...	
72	lw \$4,50(\$7)

Salto efectivo





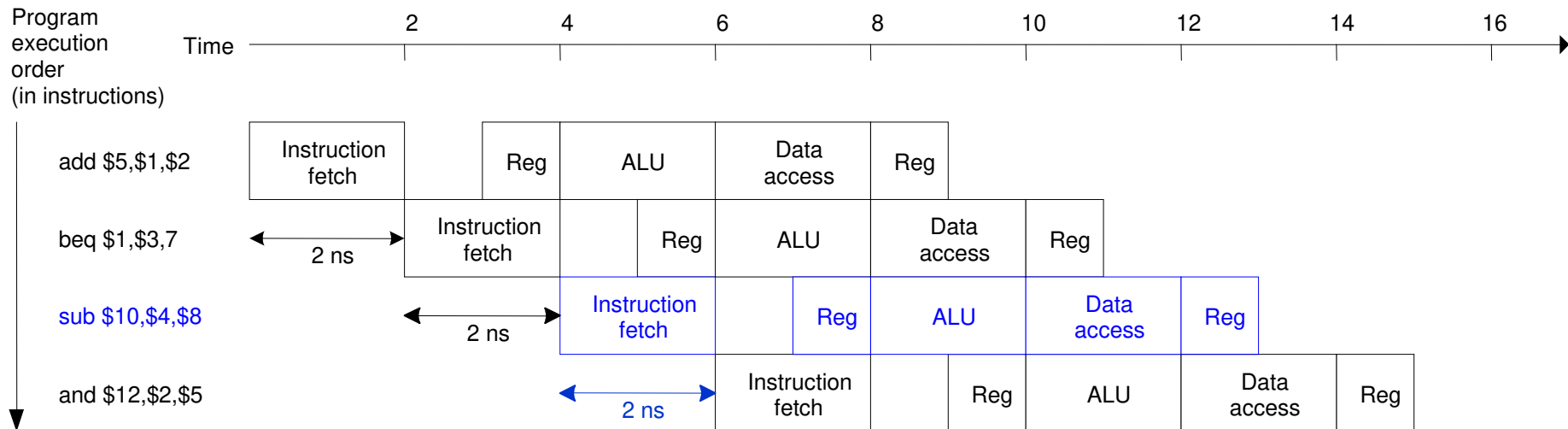
Salto retardado: ejemplo

Rellenamos el hueco con la instrucción anterior.

El traductor intercambia las instrucciones **sub** y **beq** (y modifica el desplazamiento en esta última)

32	add \$5, \$1, \$2
36	sub \$10, \$4, \$8
40	beq \$1, \$3, 7
44	and \$12, \$2, \$5
48	or \$13, \$2, \$6
52	add \$14, \$4, \$2
56	slt \$15, \$6, \$7
...	
72	lw \$4, 50(\$7)

Salto no efectivo





Riesgos y rendimiento en la segmentación

Los riesgos reducen el rendimiento de la segmentación:

$$A_{n_i} = \frac{\text{Tiempo sin segmentación}}{\text{Tiempo con segmentación}} = \frac{t_{\text{noseg}} * n_i}{t_{\text{seg}} * (n_e - 1 + n_i + \text{total de ciclos gastados en burbujas})}$$

donde

- t_{noseg} : tiempo medio de ejecución por instrucción en una máquina no segmentada.
- t_{seg} : duración del ciclo de reloj en la máquina segmentada.
- n_i : número de instrucciones ejecutadas
- n_e : número de etapas de la segmentación



6. Segmentación del camino de datos

Una vez leída una instrucción, las fases siguientes a IF en adelante necesitan cierta información proporcionada por las fases anteriores para llevar a cabo su tarea:

- ID: necesita instrucción y PC incrementado.
- EX: necesita registros, PC incrementado y posible desplazamiento.
- MEM: necesita resultado UAL y código de condición, registro fuente, dirección de salto.
- WB: necesita resultado UAL y dato leído de memoria.

Si en cada ciclo entra una instrucción nueva, la información generada por la instrucción previa en su fase de IF se pierde, a no ser que la guardemos en un registro.

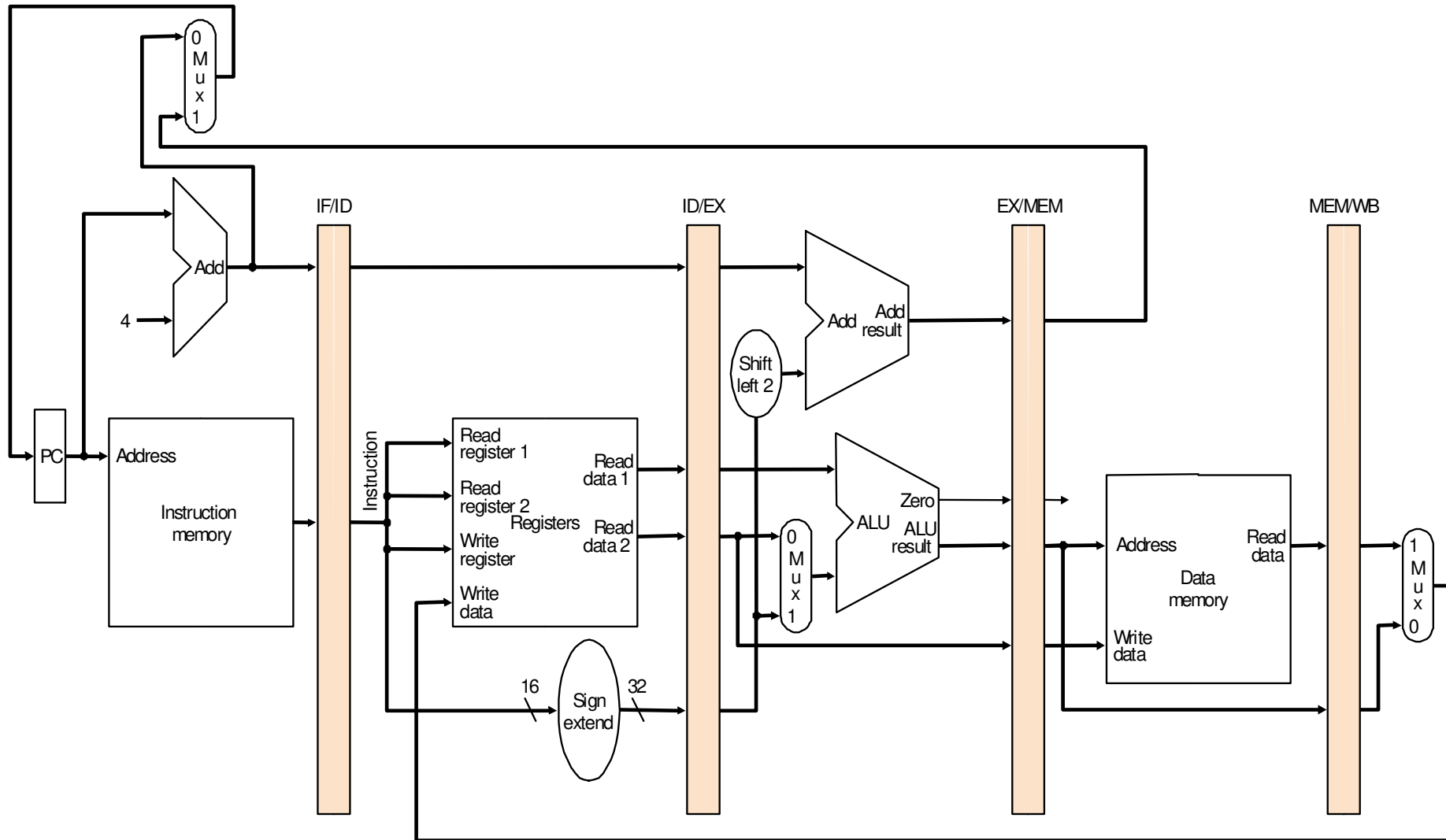
- Lo mismo sucede con la información generada por las siguientes fases.

Registros de la segmentación: almacenan información producida por fases anteriores y la hacen disponible para la realización de las fases posteriores.

- Hay un registro entre cada dos etapas de la segmentación.
- Los registros de la segmentación se cargan con nueva información en cada ciclo de reloj.



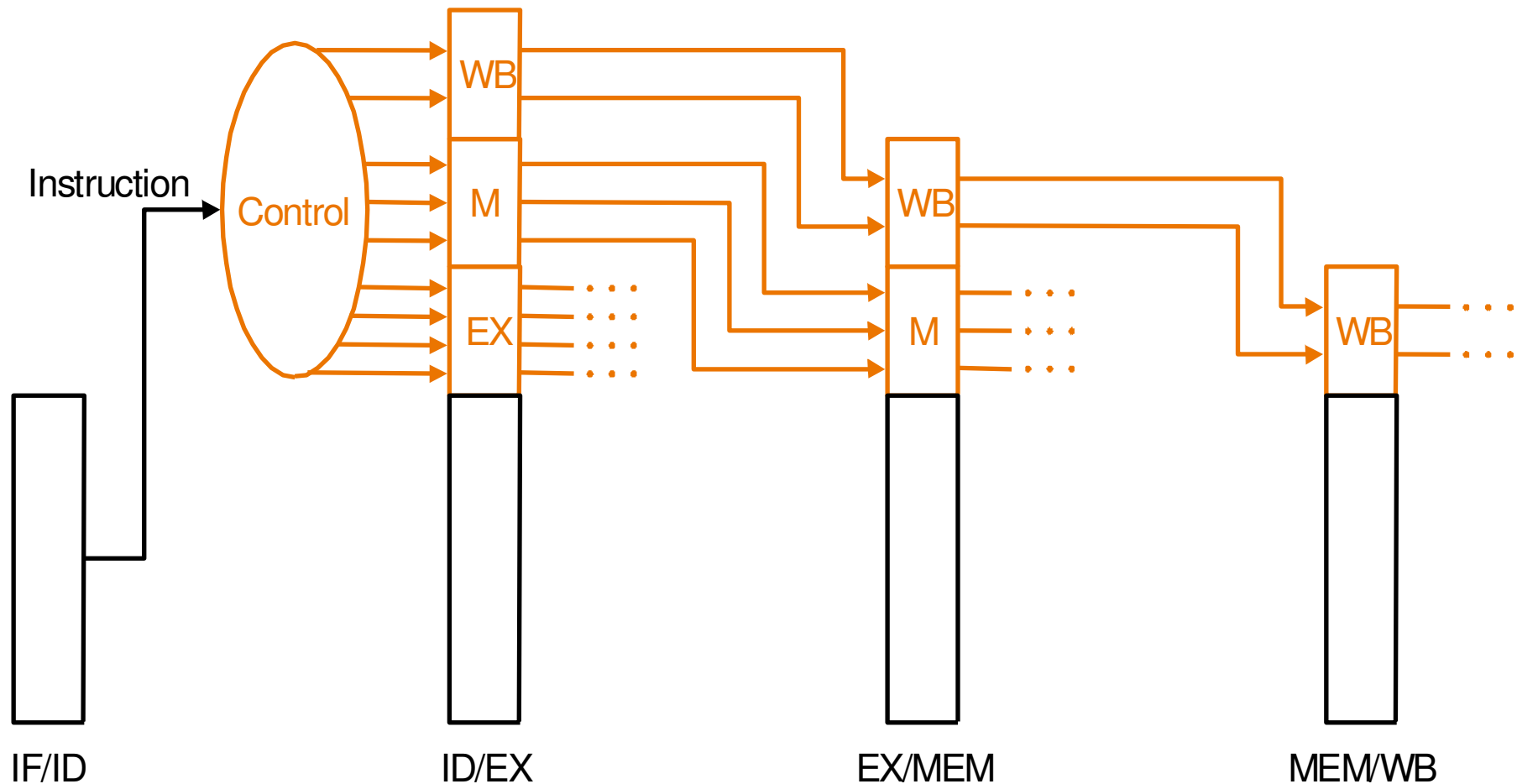
Segmentación del camino de datos





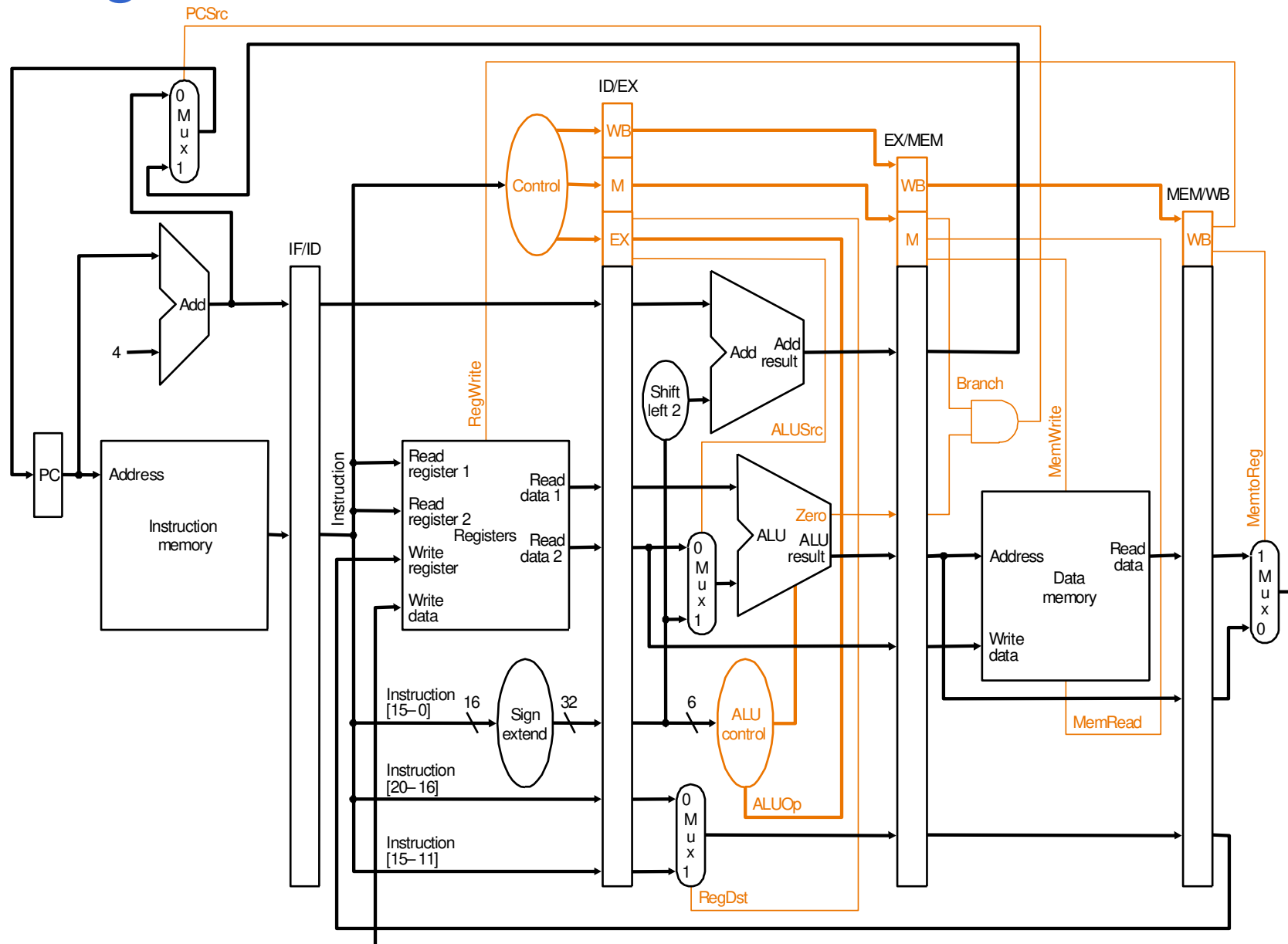
Segmentación del camino de datos

Los registros de la segmentación también se utilizan para transportar la información de control desde la fase ID hasta las demás.





Segmentación del camino de datos: control





Segmentación del camino de datos

- Las instrucciones entran por la izquierda y recorren el cauce hacia la derecha hasta finalizar su ejecución.
- Dos excepciones:
 - La etapa de escritura de resultados que almacena el dato en el banco de registros situado en la etapa 2 => Riesgo de datos.
 - La selección del valor del PC, que se elige en la etapa MEM entre el PC incrementado y la dirección de salto => Riesgo de control.
- El camino de datos anterior debe ser modificado cuando menos para:
 - Detectar y controlar los riesgos
 - Reducir o eliminar las burbujas producidas por los mismos.
 - Gestionar excepciones.

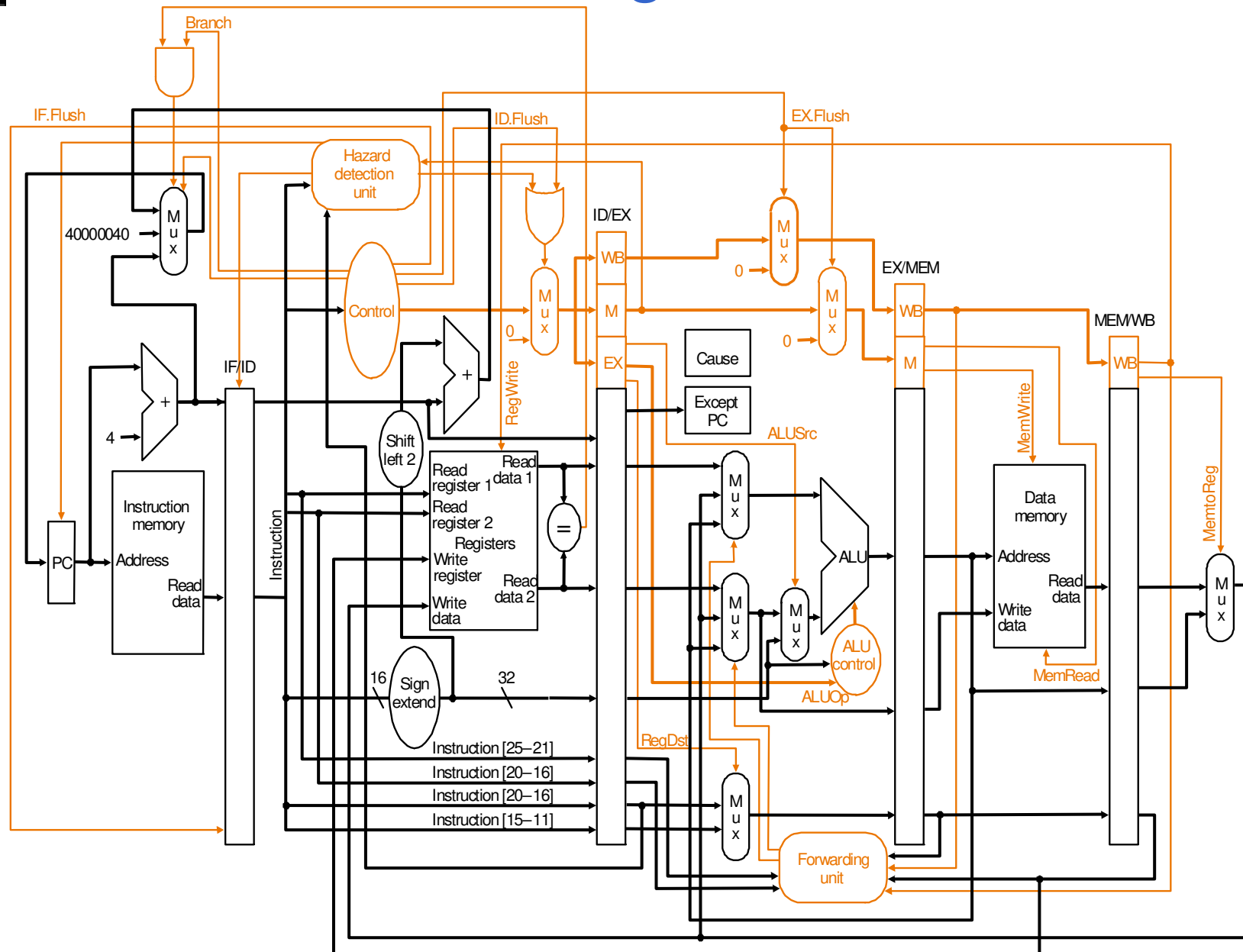


Segmentación del camino de datos: mejoras

1. **Obtención de la información de salto:** moverla a la etapa ID.
2. **Circuitería para anticipación:**
 - **Unidad de anticipación** (*forwarding unit*): detecta cuándo es necesario.
 - Enlaces y otros elementos para realizar la anticipación.
3. **Circuitería para detección y tratamiento de riesgos:**
 - **Unidad de detección de riesgos** (*hazard detection unit*).
 - Señales (IF.Flush, carga del PC y del registro IF/ID) y elementos utilizados para bloquear la segmentación.
4. **Circuitería para excepciones:**
 - Señales ID.Flush y EX.Flush y elementos asociados.



Camino de datos segmentado: versión final



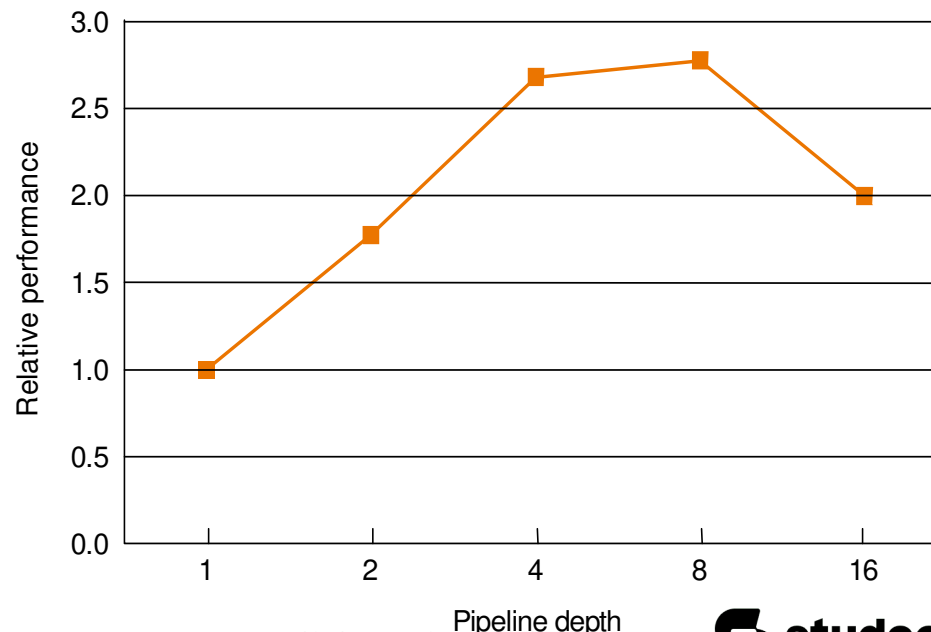


7. Técnicas avanzadas de segmentación

⇒ **Supersegmentación:** división del cauce en muchas etapas (8 ó más).

⇒ El aumento del rendimiento con la segmentación está íntimamente relacionada con el número de etapas del cauce $\Rightarrow$ ¿Por qué no dividir el cauce en más etapas?

1. Poner más etapas dificulta la gestión de riesgos.
2. Si el ciclo de reloj es muy corto:
 - El tiempo de carga de registros es significativo.
 - También empieza a ser significativo el sesgo del reloj (*clock skew*).





8. Conclusiones e implicaciones

- ⇒ La segmentación del camino de datos y el control proporciona un rendimiento mucho más elevado que las versiones uniciclo y multiciclo.
 - En cada instante, todos o casi todos los elementos del camino de datos permanecen ocupados realizando trabajo útil.
- ⇒ El control de una máquina segmentada es complejo.
 - Se complica el manejo de excepciones e interrupciones.
- ⇒ Al solaparse la ejecución de las instrucciones se producen ciertos problemas:
 - Riesgos estructurales.
 - Riesgos de datos.
 - Riesgos de control.
- ⇒ Los riesgos pueden producir bloqueos en el cauce que hacen que disminuya el rendimiento.
- ⇒ Las características del repertorio de instrucciones de una máquina son cruciales a la hora de implementarla en forma segmentada.



Conclusiones e implicaciones

Tendencias en los repertorios de instrucciones:

⇒ RISC: Reduced Instruction Set Computer (computador con repertorio de instrucciones reducido).

- Instrucciones sencillas, homogéneas y poco potentes.
- Pocos modos de direccionamiento.
- Pocos formatos de instrucción.
- Codificación uniforme, con códigos de operación de tamaño y posición fijas.
- Ejemplo: MIPS (*Microprocessor without Interlocked Pipeline Stages*).

⇒ CISC: Complex Instruction Set Computer (computador con repertorio de instrucciones complejo).

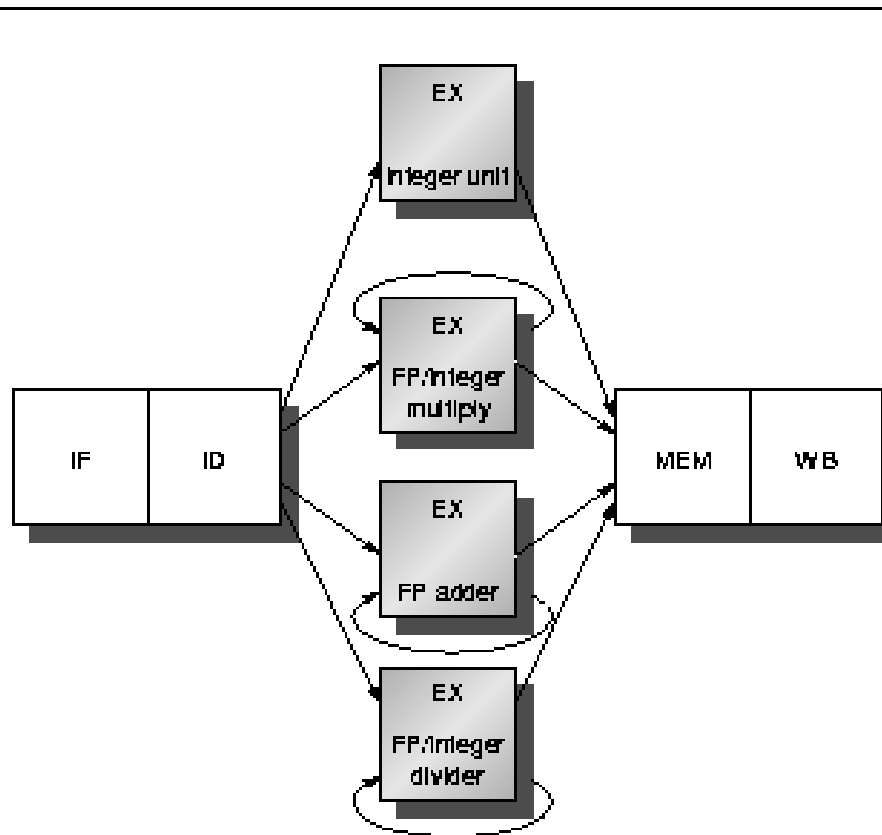
- Instrucciones complejas, no homogéneas, algunas de ellas muy potentes.
- Muchos modos de direccionamiento.
- Muchos formatos de instrucción con codificaciones complejas.
- Ejemplo: Intel 80x86, Pentium, familia del Motorola 68000.

⇒ Es mucho más fácil segmentar una máquina RISC.



Conclusiones e implicaciones

- ⇒ Hay operaciones que no pueden completarse en un solo ciclo.
 - Las más significativas son las de coma flotante.
 - Para aumentar el rendimiento se añaden varias unidades funcionales especializadas.





Conclusiones e implicaciones

- ⇒ Algunas unidades necesitan varios ciclos de ejecución:
 - Unidades segmentadas (suma y multiplicación FP).
 - Unidades no segmentadas (divisor) => parones estructurales.

- ⇒ Para cada unidad hay que dar:
 - Latencia: N° de ciclos vacíos que hay que esperar entre la emisión de la instrucción que produce el resultado y la siguiente que lo utiliza (emisión = paso de una instrucción a la fase EX).
 - Intervalo de repetición: N° de ciclos entre la emisión de dos operaciones del mismo tipo.

- ⇒ ¡ Las instrucciones se emiten en orden, pero pueden finalizarse en desorden !
 - Pueden surgir riesgos de tipos WAR y WAW.



Conclusiones e implicaciones

